# AlgoCamp AI Engineering Notes — The ReAct Pattern & Building Coding Agents

> *AI Engineering cohort, session 4. Notes on the ReAct (Reasoning + Acting) pattern, why LLMs need tools, the Thought  Action  Observation loop, reasoning models (o1/R1) vs traditional LLMs, ReAct vs CoT vs Act, the tools a coding agent needs, human-in-the-loop safety, sandboxing, long-running-task challenges, open-source agents (Cline, Pi, DeepWiki), frameworks (LangChain, CrewAI), and the plain-Python agent-loop homework.*

---

## 1. From Harness to Agent

Last session ended on **harness engineering** — building an ecosystem around the LLM (tools, evals, sub-agents). An **agent** is exactly that: an LLM wrapped in a loop that lets it **reason** about a task and **act** on the world through tools. This session is the foundational pattern that makes agents work: **ReAct**.

> *Framing AlgoCamp reiterated: this work lives in the **application + inference layers**, not the LLM layer. You use **fixed model weights** as-is (inference) and build systems around them — you're not modifying the model.*

---

## 2. The Core Problem — LLMs Can't *Do* Anything

An LLM only generates **token sequences**. That's its entire power and its entire limitation. It **cannot**, on its own:

- read or write files
- run terminal commands
- query a database or code repository
- search the web or hit an API

```
LLM alone:   text in    text out.  Nothing else.
             It can DESCRIBE running a command, but it can't RUN one.
```

To be useful for real tasks, the model needs a way to **interface with the environment** — and since it only emits tokens, the trick is: **let it emit tokens that signal which tool to call.** A surrounding program reads those tokens, executes the tool, and feeds the result back. That's the essence of tool use, and ReAct is the pattern that structures it.

---

# 3. The ReAct Pattern (Reasoning + Acting)

## The paper

**ReAct** ("ReAct: Synergizing Reasoning and Acting in Language Models," Yao et al., 2022) proposes interleaving **reasoning** (thinking through the problem) with **acting** (taking actions via tools) — in a loop — rather than doing either alone.

## The synergy

- **Reasoning alone** (Chain-of-Thought) — the model thinks, but works only from its frozen internal knowledge  can hallucinate facts it doesn't have.
- **Acting alone** — the model takes actions, but without reasoning it flails, with no plan tying actions together.
- **ReAct** — reasoning *guides* which action to take next; the action's *result* informs the next reasoning step. Each improves the other.

> Reasoning tells you WHAT to do  Acting gets you real information
> new information improves your reasoning  better next action  ...

## Why it reduces hallucination

Because the agent can **fetch real facts** (via a search/tool action) instead of guessing from memory, and then *reason over the retrieved fact*, ReAct **reduces hallucination and error propagation** — a wrong guess doesn't compound because the next step grounds itself in a real observation.

---

# 4. The ReAct Loop — Thought / Action / Observation

The loop the agent runs, over and over, until it can answer:

```
THOUGHT:      reason about what to do next
ACTION:       call a tool (with arguments)
OBSERVATION:  read the tool's result
   back to THOUGHT with the new info
... repeat until ...
ANSWER:       produce the final response
```

### The paper's canonical example (Apple Remote / Front Row)

The ReAct paper illustrates this with a multi-hop question: *"Aside from the Apple Remote, what other device can control the program the Apple Remote was originally designed to interact with?"*

```
Thought:  I need to find what program the Apple Remote was designed to control.
Action:   Search[Apple Remote]
Obs:      The Apple Remote was designed to control the Front Row media program.

Thought:  Now I need what OTHER device controls Front Row.
Action:   Search[Front Row software]
Obs:      Front Row can be controlled by the Apple Remote or the keyboard function keys

Thought:  The answer is the keyboard function keys.
Answer:   Keyboard function keys.
```

A pure CoT model might **hallucinate** the answer; a pure Act model might search aimlessly. ReAct reasons about *what to search*, reads the real result, and reasons again — arriving at a grounded answer. (The paper interfaces with the Wikipedia API for these Search/Lookup actions.)

---

## 5. Reasoning Models (o1, R1) vs Traditional LLMs

An important distinction AlgoCamp drew:

- **Traditional LLM** — pure next-token prediction; answers "immediately," no explicit deliberation.

- **Reasoning model** (OpenAI **o1**, DeepSeek **R1**) — trained to generate **intermediate reasoning tokens** *before* the final answer. It breaks a complex problem into sub-problems, explores multiple paths, and evaluates them, then produces the answer.

```
Traditional:  question    answer                (fast, can be shallow)
Reasoning:    question    [lots of internal
                          reasoning tokens]     answer  (slower, more robust)
```

### How this relates to ReAct

Reasoning models make the **"Thought"** step of the ReAct loop much stronger — they natively deliberate before acting. But note: **reasoning is internal thinking; ReAct is external interaction.** A reasoning model still needs tools (the Action/Observation part) to touch the world. They're complementary: reasoning improves *what to do*, ReAct lets the agent *actually do it and observe the result*.

---

## 6. ReAct vs Chain-of-Thought vs Act — Performance

The paper compared strategies across QA and fact-verification tasks:

| Strategy | Behavior | Weakness |
|---|---|---|
| **Act** (act only) | tool calls, no reasoning | flails; no plan |
| **CoT** (reason only) | internal reasoning, no tools | hallucinates; can't get new facts |
| **ReAct** | interleaves both | best grounding, fewer hallucinations |

Findings AlgoCamp highlighted:

- **ReAct consistently outperforms Act** — reasoning about actions beats blind acting.
- **ReAct is strong across domains** — question answering, fact verification, decision-making.
- A nuance on *how you deploy the model*: **ReAct tends to shine when the model is fine-tuned** for it, while for **prompting** large models, **Chain-of-Thought** (or combining ReAct + CoT) is often strongest. The practical takeaway: **combining reasoning and acting** beats either alone; the best mix depends on whether you're prompting or fine-tuning.

---

## 7. Tools for a Coding Agent (Claude Code-like)

The goal project: a coding agent like Claude Code. What tools does it need?

```
FILE SYSTEM (CRUD)        COMMAND EXECUTION       EXTRAS

create_file               run_terminal_command    web_search
read_file                 (build, test, install)  bug_detection across repo
edit_file                                         code navigation
delete_file
list_files
```

- **File operations** — the agent reads code to understand it, writes code to change it, lists directories to navigate.
- **Command execution** — run builds, tests, installs, git — the actions that make a coding agent *do* things, not just suggest.
- **Extras** — web search (docs, errors), bug detection across a repository, etc.

Each tool is a function the agent can invoke by emitting a structured action; the harness executes it and returns the observation (§2, §4).

---

## 8. Human-in-the-Loop (HITL) — The Safety Layer

### Why it's essential

A coding agent that can run `rm -rf` or delete files **autonomously** is dangerous. **Human-in-the-loop** inserts an **approval step** before **side-effecting actions** execute.

```
Agent proposes:  run_terminal_command("rm -rf build/")
    PAUSE    show the human the proposed action
    human APPROVES or REJECTS
    only on approval does it execute
```

### What needs approval

- **Destructive / side-effecting commands** — deleting, overwriting, network calls, installs.
- **Sensitive access** — e.g. **password-protected files** requiring human intervention.

- Read-only actions (list, read) can often run freely; it's the **actions with consequences** that gate on a human.

### The principle

Modern coding agents should **verify before executing**, not execute autonomously and apologize later. The human is the circuit breaker for anything irreversible or risky. (This is why Claude Code, Cline, etc. ask before running commands.)

---

## 9. Isolation & Sandboxing

Related safety measure: **run agent-executed code in isolation** — a sandbox / container — especially for testing. Reasons:

- Untrusted or AI-generated code shouldn't touch your real system.
- A mistake (or a malicious instruction) is contained within the sandbox.
- Security: **prevent the agent from accessing sensitive data** outside its allowed scope.

Combined with HITL, sandboxing gives you two layers: *approve* what runs (HITL) and *contain* where it runs (sandbox).

---

## 10. Long-Running Task Challenges

Coding tasks can run long (many steps, big repos). Problems that emerge:

- **Context management** — a long task overflows the context window (session 2); the agent "forgets" earlier steps. Mitigations: **summarize** progress, **slide** the window.
- **Failures / timeouts** — a step crashes or hangs; the agent needs to recover, not die.
- **State across sessions** — resuming work later.

### Practical solutions discussed

- **Share session history** between chats so context carries over.
- **Track progress in external files** (e.g. a `.md` progress/todo file the agent reads and updates) — externalizing state so it survives context limits and restarts.

```
progress.md:
 [x] set up project
 [x] write file CRUD tools
 [ ] add terminal execution      agent reads this to know where it left off
 [ ] add HITL approval
```

> *This is where **message queues / Temporal** come in (next session) — durable orchestration for long-running agent tasks that must survive restarts and failures. The booking-project Temporal knowledge transfers directly.*

---

# 11. Open-Source Agents & Frameworks

## Reference agents to study

- **Cline** *(recap: "Klein")* — a popular open-source coding agent; good reference architecture.
- **Pi** — another referenced coding agent to explore.
- **DeepWiki** — a tool for **understanding repositories** (AI-generated wikis/architecture views of a codebase) — useful for reverse-engineering how these agents are built.

> *Studying real open-source agents is the fastest way to learn agent architecture — see how they define tools, structure the loop, and implement HITL.*

## Frameworks for building agents

- **LangChain** — the framework the cohort will use to build the coding agent (starting next week).
- **CrewAI** — multi-agent orchestration framework (agents collaborating).
- **AgentX** — another referenced alternative.
- **Temporal** — not an agent framework, but the durable-execution engine for **long-running** agent tasks (message-queue role).

The plan: build an MVP coding agent in **plain Python first** (homework), then rebuild with **LangChain**, then add long-running tasks + sandboxing.

---

# 12. Homework — A Plain-Python Agent Loop

Before frameworks, build the loop by hand to understand it. The assignment: a simple Claude Code-like agent with **file-system CRUD** + **terminal command execution**, with **security** in mind.

## The minimal agent loop

```
import json, subprocess, os
from openai import OpenAI

client = OpenAI()

# 1. Define tools the model can call
TOOLS = {
    "read_file":  lambda path: open(path).read(),
    "write_file": lambda path, content: open(path, "w").write(content),
    "list_files": lambda path=".": os.listdir(path),
    "delete_file": lambda path: os.remove(path),
    "run_command": lambda cmd: subprocess.run(cmd, shell=True,
                                    capture_output=True, text=True).stdout,
}

SIDE_EFFECTING = {"write_file", "delete_file", "run_command"}  # need approval

SYSTEM_PROMPT = """You are a coding agent. To use a tool, respond with JSON:
{"thought": "...", "action": "tool_name", "args": {...}}
When done, respond with {"thought": "...", "answer": "..."}"""

def run_agent(task: str):
    messages = [
        {"role": "system", "content": SYSTEM_PROMPT},
        {"role": "user", "content": task},
    ]

    while True:  # the ReAct loop
        resp = client.chat.completions.create(model="gpt-4", messages=messages)
        raw = resp.choices[0].message.content
        step = json.loads(raw)
        messages.append({"role": "assistant", "content": raw})

        # THOUGHT is in step["thought"] (reasoning)
```

```
    if "answer" in step:                    # ANSWER    done
        return step["answer"]

    action, args = step["action"], step.get("args", {})

    # HUMAN-IN-THE-LOOP for side-effecting actions
    if action in SIDE_EFFECTING:
        print(f"Agent wants to run: {action}({args})")
        if input("Approve? (y/n): ") != "y":
            observation = "REJECTED by human"
        else:
            observation = str(TOOLS[action](**args))
    else:
        observation = str(TOOLS[action](**args))   # ACTION

    # OBSERVATION fed back into context
    messages.append({"role": "user", "content": f"Observation: {observation}"})
```

### What to notice

- The **loop** is literally Thought  Action  Observation  repeat (§4).
- **HITL** gates the dangerous tools ( `SIDE_EFFECTING` ) — approve before executing (§8).
- The model **signals tools via structured output** (JSON) — the harness parses and executes (§2).
- **Security to add:** restrict file paths to a working directory (no `../../etc/passwd` ), run commands in a **sandbox** (§9), never expose secrets to the agent.

Build this, feel the loop, *then* appreciate what LangChain automates for you.

---

## What's Coming

- Next session: **message queues + Temporal basics**, then start building the coding agent with **LangChain**.
- AlgoCamp to share: the session's **GitHub repo**, a formal **homework** spec, and an intro to open-source agents (Cline, Pi) + **DeepWiki**.

---

## Action Items

**AlgoCamp:**

- Post the formal homework (plain-Python coding agent: file CRUD + terminal + HITL).
- Share the session GitHub repo.
- Prepare an intro to open-source agents (Cline, Pi) and DeepWiki.
- Next session: message queues, Temporal basics, LangChain coding agent.

**Attendees:**

- Build a simple coding agent with a **plain Python agent loop** — file-system CRUD + terminal execution — with **security** and **HITL approvals** in mind.

---

## Concept Cheat-Sheet

- **LLMs only emit tokens** they can't touch files/terminals/APIs directly; tools bridge the gap by letting the model *signal* a tool call the harness executes.
- **ReAct** (Yao et al., 2022) interleave **Reasoning** + **Acting**; reasoning picks the action, the observation improves the reasoning.
- **The loop:** Thought Action Observation (repeat) Answer.
- **Reduces hallucination** by grounding reasoning in real tool results, stopping error propagation.
- **Reasoning models (o1, R1)** generate intermediate reasoning tokens before answering; strengthens the "Thought" step but still need tools to act.
- **ReAct vs CoT vs Act** ReAct beats Act consistently; combining reasoning + acting wins; ReAct shines when fine-tuned, CoT strong for prompting.
- **Coding-agent tools:** file CRUD + command execution + web search / bug detection.
- **HITL** human approves side-effecting actions (delete, run, install, sensitive files) before they execute; verify, don't auto-run.
- **Sandbox** run agent code in isolation; contain mistakes, protect sensitive data.
- **Long-running tasks** manage context (summarize/slide), track progress in external `.md`, use Temporal for durability.
- **Study:** Cline, Pi (open-source agents), DeepWiki (repo understanding). **Build with:** LangChain (also CrewAI, AgentX); Temporal for long tasks.
- **Homework:** hand-build the ReAct loop in plain Python before using a framework.

---